

Reviewer Pack — Euler Characteristic of Matroid Cycles Bounds Communication ...

Entry fbed9108e692 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 01:05:57 UTC

Euler Characteristic of Matroid Cycles Bounds Communication Complexity

Entry ID: fbed9108e692 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 01:05:57 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Communication Complexity

Statement:

For all Boolean matrices M with at most 40 rows, the communication complexity of computing the determinant of M is $O(n^2 - \chi(M))$, where n is the number of columns and $\chi(M)$ is the Euler characteristic of the matroid associated with M .

Rationale (proposer's reasoning):

Matroid theory provides a combinatorial structure that generalizes the concept of linear independence, and it has been applied to various areas of mathematics. The conjecture suggests that the communication complexity of computing determinants can be linked to the topological properties of matrices, potentially revealing new connections between algebraic geometry and communication complexity.

Taxonomy category: MATROID_COMMUNICATION_COMPLEXITY_BOUND (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f799e71d260bf18b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient between the communication complexity and $n^2 - \chi(M)$ is ≥ 0.7 , for at least 80% of seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -matroid theory AND communication complexity - Euler characteristic AND determinant AND communication complexity - matroid cycles AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1608.04025v1] Quasi-matroidal classes of ordered simplicial complexes - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2509.13173v1] Euler’s explorations of extremal ellipses - [http://arxiv.org/abs/1808.02841v1] On divergent Series - [http://arxiv.org/abs/math/0610652v1] Letters relating to a Theorem of Mr. Euler, of the Royal Academy of Sciences at Berlin, and F.R.S. for correcting the Ab - [http://arxiv.org/abs/1810.06267v2] Small Space Stream Summary for Matroid Center - [http://arxiv.org/abs/1510.04532v2] Internally Perfect Matroids

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            # Find pivot
            max_row = i
            for k in range(i+1, n):
                if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                    max_row = k
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

            # Eliminate above and below
            pivot = matrix[i][i]
            for j in range(n):
                matrix[i][j] /= pivot
            for k in range(n):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(n):
                        matrix[k][j] -= factor * matrix[i][j]
        return matrix
```

```

def determinant(matrix):
    n = len(matrix)
    det = 1
    for i in range(n):
        det *= matrix[i][i]
    return det

def eulertour(graph, start):
    visited = set()
    stack = [start]
    euler_tour = []
    while stack:
        u = stack[-1]
        if u not in graph or len(graph[u]) == 0:
            euler_tour.append(u)
            stack.pop()
        else:
            v = graph[u].pop()
            graph[v].remove(u)
            stack.append(v)
    return euler_tour

def matroid_euler_characteristic(matrix):
    n = len(matrix)
    m = len(matrix[0])
    graph = {i: set() for i in range(n)}
    for i in range(n):
        for j in range(m):
            if matrix[i][j] == 1:
                graph[i].add(j)

    euler_tour = eulertour(graph, 0)
    return len(euler_tour) - n

def communication_complexity(matrix):
    n = len(matrix)
    m = len(matrix[0])
    det = determinant(gaussian_elimination(matrix))
    chi = matroid_euler_characteristic(matrix)
    return abs(det), n**2 - chi

instances_tested = 0
metric_sum = 0.0
max_n = 0
conjecture_holds = True
counterexample = ""

for n in [5, 10, 15, 20, 30, 40]:
    if instances_tested >= 30:
        break

    matrix = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    comm_complexity, chi = communication_complexity(matrix)

    metric_sum += comm_complexity

```

```

instances_tested += 1
max_n = n

if comm_complexity > n**2 - chi:
    conjecture_holds = False
    counterexample = f"Matrix with n={n} and det={comm_complexity}, chi={chi}"

return {
    "metric_name": "communication_complexity",
    "metric_value": metric_sum / instances_tested,
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4313c397.py", line 119, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4313c397.py", line 94, in run_trial
    comm_complexity, chi = communication_complexity(matrix)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4313c397.py", line 79, in communication_complexity
    det = determinant(gaussian_elimination(matrix))
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4313c397.py", line 34, in gaussian_elimination

```

```
matrix[i][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, preventing any evaluation of the conjecture's support conditions. | next: Investigate and fix the division by zero error in the code to allow for proper testing of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	25704
2	preregistration	ollama_remote	glm4:latest	0	0	26362
3	novelty	ollama_remote	glm4:latest	0	0	23993
4	novelty	ollama_remote	glm4:latest	0	0	18193
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16294
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16124
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11075
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15815
9	judge	ollama_remote	glm4:latest	0	0	11870

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 165431 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fbed9108e692.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fbed9108e692.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fbed9108e692.tar.gz` (if generated)